MMA 616 · MANAGING INTELLIGENCE

# Introduction + Day 1: Plan

You can already build almost anything. This course is about what is worth building.

Dr. Vern L. Glaser (vglaser@ualberta.ca) · TA Jennifer Kotadia (jkotadi1@ualberta.ca)

TODAY

# Day 1 at a glance, four blocks

**Block 1**

## Why judgment is the constraint

The moment, and the method: Agentic Project Design

**Block 2**

## Knowledge: stock and curate

Connect the source and curate it, then present your data and knowledge

**Block 3**

## The living spec

Choose what's worth building, author the CLAUDE.md, then present it

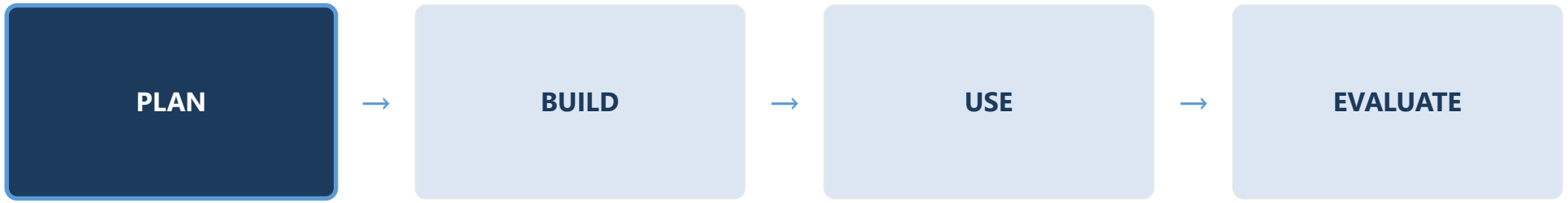
**Block 4**

## Make the spec visible

Mock the deliverable in HTML, then present your mockup

WHERE WE ARE

# Today's position in the loop




You run this loop for four days. Day 1 is Plan: decide what is worth building, then spec it.

# WHY

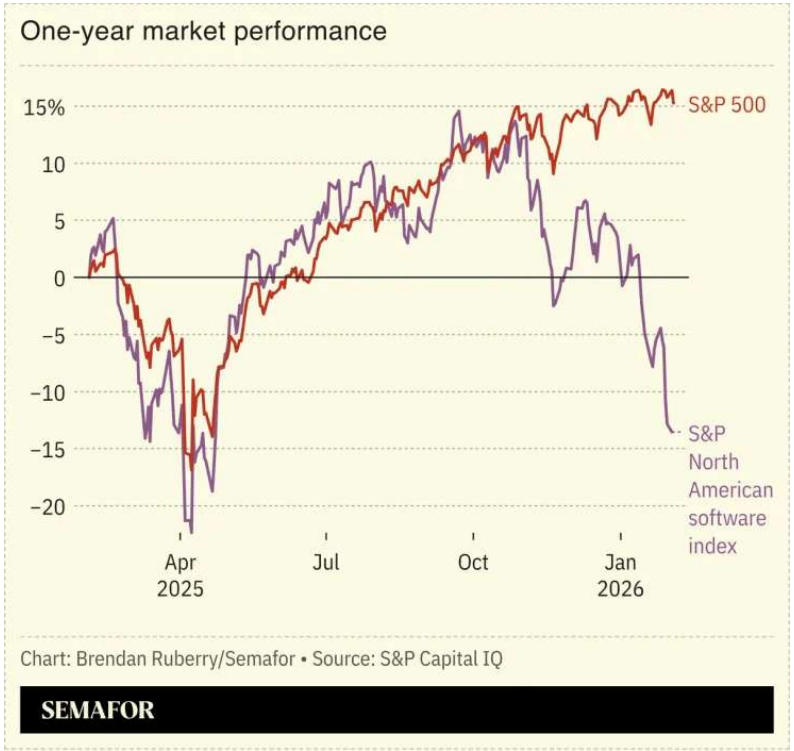
INTRODUCTION

## Why learn to vibe code, and the catch

Building is becoming easy. Knowing what to build, and whether it is any good, is the hard part.

FEBRUARY 2026

# The SaaSpocalypse




*Roughly 285 billion dollars erased from software firms in about two days as agents began doing white-collar work end to end.*

WHAT THE SELLOFF REVEALED

# If building is nearly free, why rent software?

- **The calculus tilts toward build:** buy-versus-build now favors building it yourself.
- **Leverage is the obvious reason** to learn vibe coding, and the smaller half of it.
- **The larger half:** being able to build tells you nothing about whether it is worth building.
- **The trap:** "I can build it, therefore it is valuable."

> ***The most costly AI failures are capable builds aimed at the wrong target.***

THE EVIDENCE

# A bottleneck that moved upstream

**~25%**

of AI initiatives delivered the return they promised

*IBM, 2025*

**16%**

had scaled across the enterprise

*IBM, 2025*

**60%**

of companies generating no material value from AI

*Boston Consulting Group, 2025*

***These describe a bottleneck that moved off the technology and onto the judgment around it.***

THE EMBLEM

# Klarna: The Bot That Hit Every Number

February 2024, the AI assistant handled 2.3 million conversations a month, two-thirds of support volume, the work of 700 agents, with resolution time down from eleven minutes to under two (Klarna, 2024).

Fifteen months later the company reversed course and began rehiring. Nothing broke; the system optimized exactly what it was told, cost and speed, while accuracy and trust were never on the board (Fortune, 2025).

> ***The failure was not capability; it was judgment about what to build and what "good" had to mean.***

*What's scarce is knowing what to ask for.*

**Mollick, 2026**

WHAT IS AN AGENT?

# Model, tools, and an orchestration loop

- **The model is the decision-maker,** an LLM such as Claude that reasons and chooses.
- **The tools are its reach:** query data, run code, write a file.
- **The orchestration layer is the loop** that feeds each result back in.
- **"LLMs using tools based on** environmental feedback in a loop" (Anthropic, 2024).
- **A workflow follows fixed steps;** an agent directs its own.

> ***At each step it reads the actual result and judges progress against it, not against its own confidence.***

EXEMPLAR · AN AGENT AT SCALE

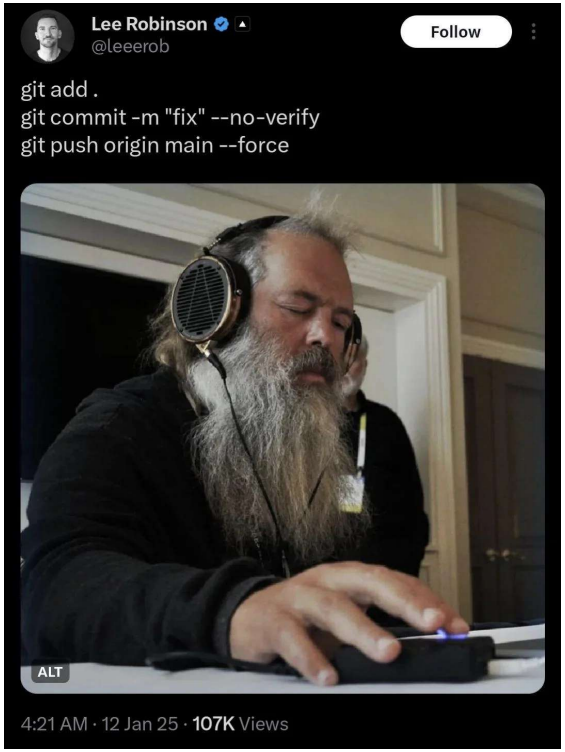
# Morgan Stanley's advisor assistant

**The business problem.** 16,000 financial advisors sat on top of a research library of about 100,000 reports, market notes, and compliance documents, far too much to search by hand while a client waited on the phone. The right answer existed; finding it fast, and consistently, did not.

**What the assistant does.** An advisor describes a client's situation in plain language; the model interprets the request, the tools search the full library, and the loop returns one answer with citations to the documents behind it. It was built to support advisors, not to replace them.

**Why it works.** The advisor, not the assistant, makes the recommendation. The agent absorbs the search and synthesis and shows its sources, so the advisor's time goes to judgment and the client, and the work can be checked before anyone relies on it.

> ***The agent does the search and synthesis; the advisor makes the decision and owns it.***




WHAT IS VIBE CODING?

# Describe what you want; steer by results

Karpathy named the practice in 2025: describe what you want in plain language, let the model write and run the code, and steer by what comes back rather than by reading every line. It lowers the barrier to building, so judgment, not coding skill, becomes the constraint. Rick Rubin, whose instrument is judgment, became its emblem with The Way of Code. Reading the code line by line is where the work stops being vibe coding; judging whether the result is right stays with you.

> ***The skill that matters shifts from writing code to judging whether the output is any good.***

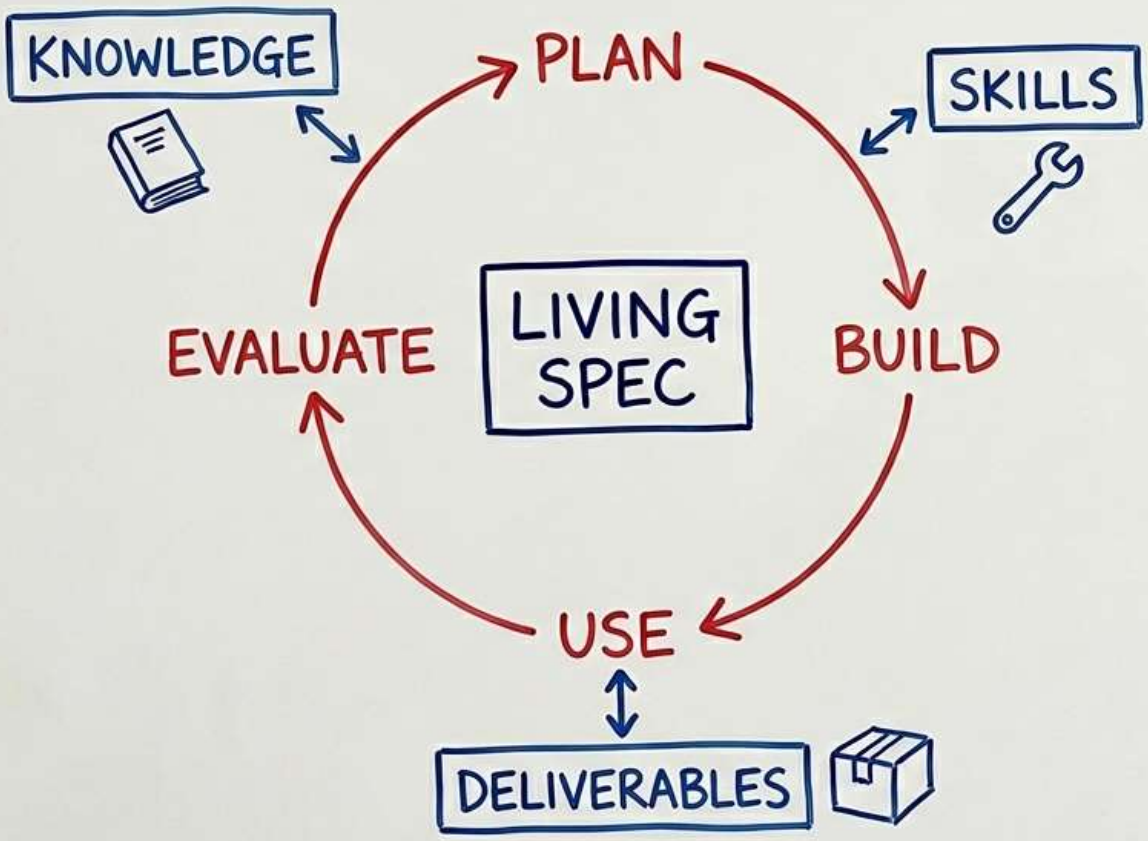
THE METHOD

# Agentic Project Design

*A method you run, not a mindset you adopt.*

# AGENTIC PROJECT DESIGN

GOVERNANCE




## PRINCIPLES

**CULTIVATE TASTE —**
know what's worth building

**WRITE WHAT GOOD LOOKS LIKE**
make standards testable

**CURATE CONTEXT —**
keep it fresh + tight

**VERIFY, THEN TRUST —**
outputs are claims, not facts

**EXPECT THE GAP —**
divergence is signal

**DELEGATE WORK, NOT ACCOUNTABILITY —**
outcomes stay yours

THE VOCABULARY

# The Six Principles

### Cultivate Taste

Know what's worth building; when in doubt, spend more on the objective.

### Write What Good Looks Like

Make standards testable; if you cannot test it, you have not written it.

### Curate Context

Keep it fresh and tight; staleness and bloat degrade output directly.

### Verify, Then Trust

Outputs are claims, not facts; read the work, not its confidence.

### Expect the Gap

Divergence is signal; instrument for it and read each gap.

### Delegate Work, Not Accountability

Agents do the work; you own the result.

THE RUNNING EXAMPLE

# The Instructor's App: Kinquiry

Across four days the instructor builds one app in parallel, on the same loop, on the STEP/KPMG Global Family Business Survey. It pairs a Data Explorer for roaming the survey with a guided Workbench: hand it a rough question and it picks the right test, runs it on the raw data, interprets the result, and renders a research presentation with the proper caveats, or refuses what the data cannot honestly answer.

On one question it reports a small but reliable association, $\beta \approx 0.16$ (95% CI [0.11, 0.20], $N \approx 2{,}180$), and then states plainly that causal interpretation is not warranted. It deliberately climbs higher than a one-week build needs to.

> ***Your realistic target is leaner, a decision-changing dashboard with a thin reasoning layer.***

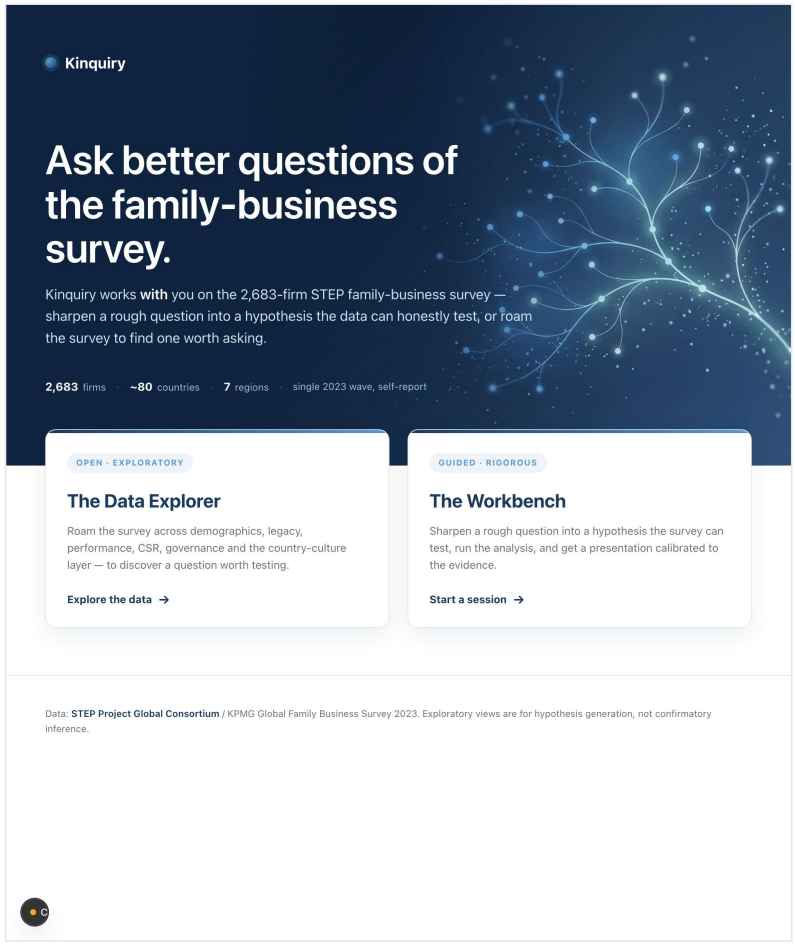
LIVE DEMO · THE GOAL

# What you are building toward

Kinquiry is the app the instructor built by running this same loop. Hand it a rough question about the family-business survey and it sharpens it into a testable hypothesis, runs the analysis, and returns a research presentation calibrated to the evidence.

By Day 4, your group ships a deployed app of your own. Two doors in: a Data Explorer for roaming the survey, and a guided Workbench for testing one question.

***Watch it work first. We will name how it was built next.***



COURSE

LOGISTICS

# How the course runs

How you are graded, and what you leave with.

THE ARC

# The four-day arc

DAY 1 · FRI JUN 26

### Plan · Cultivate Taste

Frame the opportunity, converge on one concept, author the project's living spec.

DAY 2 · SAT JUN 27

### Build · Curate Context

Turn the spec into a deployed prototype drawing on the Skills stock.

GAP · 1 WEEK

### Use

Put the app to work in real conditions; keep a structured log.

DAY 3 · FRI JUL 3

### Evaluate

Compare enacted against specified; fold the lessons back into the spec.

DAY 4 · SAT JUL 4

### The loop made legible

Capstone demos defended under questioning; launch the individual final.

LOGISTICS

### Room ESB-236 · 9–5 daily

A four-day group hackathon; ~6 groups of 4–5.

ASSESSMENT

# How you are graded

| Component | Weight | What it is |
|---|---|---|
| **Class Participation** | 20% | Engagement across discussions, hackathon work, deployment stories, peer feedback |
| **Group AI Project** | 40% | One team-built data app with an agentic layer, run through the full loop |
| **Individual Take-Home Final** | 40% | Distinct solo app on a Government of Alberta open dataset; launched Day 4 |

***The two 40% deliverables are co-equal but not run in parallel; the final proves you can run the loop alone.***

WHAT YOU LEAVE WITH

# Three things, by the end

### 1 A deployed app

Built and used for real, and evaluated against what you specified, not a demo in the room.

### 2 A repeatable loop

Run again, alone, on any data and any opportunity; what the individual final proves.

### 3 A working vocabulary

The loop, the living spec, the stocks, the governance boundary, the six principles.

***Names for where judgment has to enter, and what happens when it doesn't.***

# KNOWLEDGE

CHAPTER 2

## Stock the workspace, curate what's in it

The first physical act sets the quality ceiling.

OPPORTUNITY-FIRST IS A DISCIPLINE, NOT A TIMING RULE

# Stock the data first, still refuse to let it choose

- **The decision and the user drive the project,** not the columns that happen to be in your folder.
- **The data is a gate, not a generator:** it can veto an opportunity, it cannot invent one.
- **You read the data to ask one thing,** can it carry the opportunity you already chose?
- **Curation, not accumulation,** is the judgment Part I actually teaches.

> ***The data in your folder must not become the thing that chooses for you.***

CREATE THE WORKSPACE, STOCK THE PRIMARY MATERIAL

# Data plus the document that explains it

- **The survey file is one row per firm** of raw codes; inert until something says what they mean.
- **The codebook says what every code means;** numbers without their meaning are read confidently and wrongly.
- **That pairing, data and codebook,** is the smallest complete unit of Knowledge for a data app.
- **Connect the source, do not paste it,** so the agent reads it in full rather than a fragment.

> ***A connected source the agent reads beats a frozen fragment that happened to fit the prompt.***

TOOL · THE DELETION TEST, APPLIED TO KNOWLEDGE

# Would removing this cause a mistake? If not, cut it

- **Record only the interpretation** the agent cannot guess for itself.
- **Point at the codebook** rather than retyping what it already holds.
- **"Never average the two anchored scales" passes:** delete it and a wrong composite ships.
- **"The dataset has a region column" fails:** the agent reads that for itself.

> ***A curated five-line codebook steers; a forty-page paste steers nothing.***

LIVE DEMO

IN CLAUDE CODE

# My Knowledge workspace

*Before any build, I stock and curate the workspace, then open it in Claude Code. This is the actual folder.*

```
knowledge/
  ├─ data-cautions.md
  ├─ data-dictionary.md
  ├─ data-profile.md
  ├─ survey-data/   dataset + codebook
  ├─ reports/   STEP global reports
  ├─ research/   distilled domain notes
  └─ course-materials/
```

- **The data and the codebook that explains it,** connected so the agent reads them in full.
- **The three data notes** hold only what the agent cannot guess: anchored scales, encodings, outliers.
- **Curated, not dumped.** Every file earns its place; the rest is source material and research.

ACTIVITY

30 min · groups

# Stock and Curate Your Workspace

*Name your candidate opportunity in one sentence, then stock the Knowledge it needs.*

- Create the project workspace and connect your data source plus the document that explains it. Do not paste a fragment.
- Apply the Deletion Test: record only what the agent cannot guess (anchored scales, encodings, skip-logic, outliers); point at the codebook.
- Add a short, distilled research note on the domain, treated as claims to verify, not a dump.

**OUTPUT** *A stocked, curated workspace: the source connected, plus a tight 'what the agent can't guess' note.*

GROUP PRESENTATION · ROUND 1

~2 MIN · EACH GROUP

# Show us your data and Knowledge

*Open the workspace on screen. Show it, don't describe it.*

- **The opportunity, in one sentence:** the decision it changes, and who owns it.
- **Your data sources:** what they are, and the document that explains it.
- **Your "what the agent can't guess" note:** the two or three lines you would defend.

> ***We are listening for curation, not volume: did you record only what the agent can't guess?***

# SPEC

CHAPTER 3

# The Living Spec

Write What Good Looks Like; make standards testable.

WHAT A LIVING SPEC IS: CAUSAL, NOT DESCRIPTIVE

# The reason the app acts, not a record of it

- **One file the agent reads** at the start of every run; in Claude Code it is the CLAUDE.md.
- **Change the file and you change behavior,** with no code edit at all.
- **The test of any line:** would deleting it change the next run?
- **The load-bearing line** ("never average the two scales") often stays in heads, unwritten.

> ***A team that cannot say what its app is, for whom, and what good looks like in two pages does not yet know.***

THE VALUE TEST · ALL THREE OR NONE

# Value holds only when three things are true

### 1 A decision it changes

A beautiful analysis that changes no decision is worth nothing.

### 2 A user who owns it

An owner who both can and will act on the output; otherwise it is worth nothing.

### 3 A claim the data can support

Assert what the data cannot carry and it is worth less than nothing; someone acts on it wrongly.

***How impressive the build is sits nowhere on the list.***

FOUR MOVES · IN ORDER

# How to make the three conditions true

1 · **Frame from the work.** Name the decision, the user, and the leverage, before reaching for a column.

2 · **Read the data as a gate.** Ask one thing of it: can this data honestly carry the claim?

3 · **Name what you will refuse.** Refuse the framing a clean, correct number quietly invites.

4 · **Converge on one.** Three or more candidates, criteria locked first, the rejected one written down.

WHAT GOES IN THE FILE · SIX PARTS

# Each part carries a decision you already made

OBJECTIVE

## Move 1

The decision, the user, the leverage, the center of gravity. Finish the no-dataset sentence.

WHAT GOOD LOOKS LIKE

## The hardest part

Standards concrete enough to score, a gold example, a short must-never-produce list.

SCOPE

## Moves 2 and 3

The smallest valuable version, one stretch, and refusals as specific as the core.

DATA

## Move 2's read

What one row is, how representative, what each blank means. Point at the codebook.

CAPABILITIES

## The seam

Where the agent decides on its own, and where its behavior is fixed in advance.

HUMAN-IN-LOOP

## Governance

Where the human has the final word, and the rule that an override is logged.

LIVE DEMO

IN CLAUDE CODE

# My CLAUDE.md, the living spec

*Now I open the actual spec and read it the way the agent does. Watch where the judgment lives.*

```
# Kinquiry — Living Spec
## Objective
Turn a rough question into an
honestly hedged finding.
## What good looks like
- say "associated with," not
  "causes"; show N behind every %
## Data — do not guess
never average the two scales
```

- **The objective first,** the decision and the user, before any column.
- **"What good looks like"** written concretely enough to score against.
- **The one load-bearing line.** Delete "never average the two scales" and the next run ships a wrong number.
- **The deletion test:** if removing a line changes nothing, it is decoration.

ACTIVITY

45 min · groups

# Frame the Opportunity and Author the CLAUDE.md

*Run the four moves, converge on ONE concept, then write the living spec.*

- Frame from the work: name the decision, the user, and the leverage, and finish the no-dataset sentence, before reaching for a column.
- Read the data as a gate, name what you will refuse, then converge on one concept (3+ candidates, criteria locked, the rejected one written down).
- Write the CLAUDE.md: objective, what good looks like (testable, with a gold example), scope, data, capabilities, human-in-the-loop.
- Cold-read it on a fresh Claude or a person with none of your context; watch where they stop, and fix those lines.

**OUTPUT** *A CLAUDE.md a stranger could build from, plus the one alternative you rejected and why.*

GROUP PRESENTATION · ROUND 2

~2 MIN · EACH GROUP

# Walk us through your CLAUDE.md

*Read the file on screen. Three things we want to hear.*

- **The objective:** your no-dataset sentence, “this lets [user] do [X] they couldn't before.”
- **What good looks like:** one standard concrete enough to score, plus your gold example.
- **The load-bearing line:** the one line that, if deleted, changes the next run.

> ***We are listening for a spec a stranger could build from, with the judgment written in.***

# DELIVERABLE

CHAPTER 4

# Mockups and the HTML prototype

The spec made visible, before the build commits you.

WHY MOCK THE DELIVERABLE

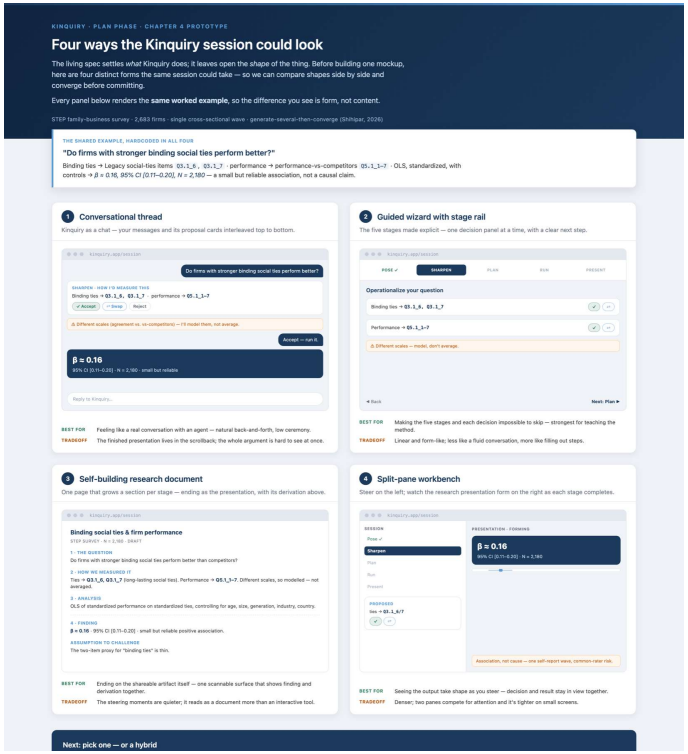
# A mockup is the spec made visible

- **Two readers handed the same sentence** picture two different documents.
- **A mockup collapses that** to one concrete picture.
- **It is a boundary object too,** surfacing disagreement now instead of in the deployment week.
- **The discipline is the walking skeleton:** the real shape with fake content.

> ***You are not making a disposable picture; you are making the app's skeleton and filling it with placeholder flesh.***

GENERATE SEVERAL, THEN CONVERGE

# Lay out distinct approaches, compare, commit




*Ask Claude for three or four distinct shapes in a grid, then converge on one; the convergence discipline applied to the deliverable's form (Shihipar, 2026).*

HTML, BY DEFAULT

# The unreasonable effectiveness of HTML

- **HTML carries more than plain text:** tables, layout, diagrams, and interactive controls (Shihipar, 2026).
- **It is more readable at scale and more shareable,** opening behind a link in any browser.
- **Reach for it** when a human must read and act on the artifact.
- **It is not free,** so spend the richness where it earns its cost.

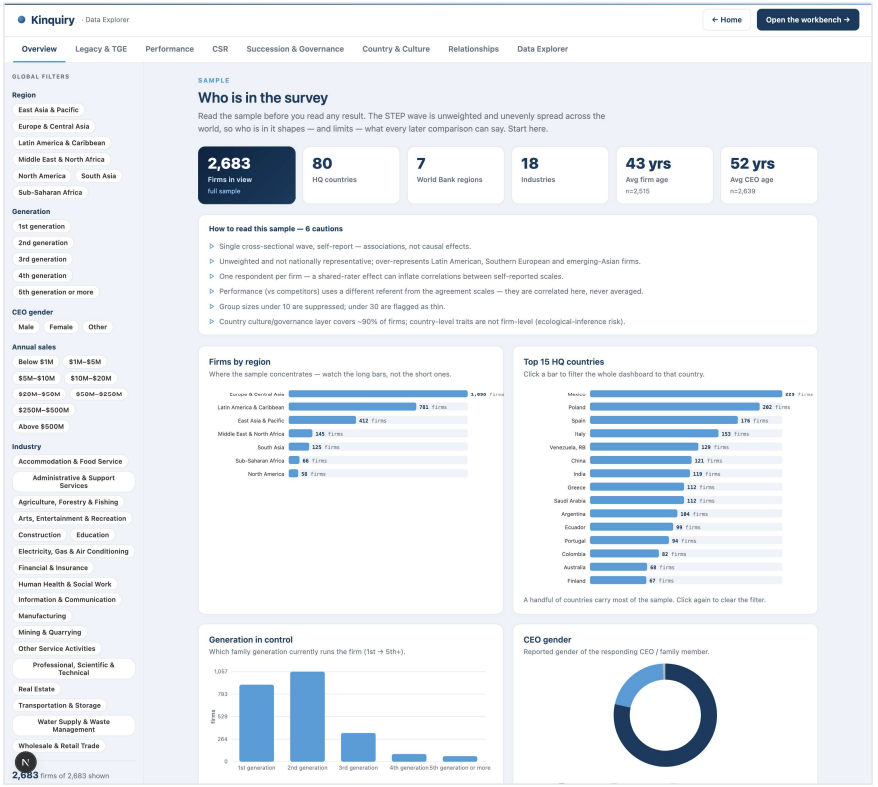
> ***Leave HTML alone when the output is a number you will read once.***

LIVE DEMO · THE DELIVERABLE

# From a mockup to the real thing

On Day 1 you mock the deliverable: the real shape, filled with placeholder content, before the build commits you to it.

Here is where it ends up. The same skeleton, now wired to the live survey: distributions, filters, and group comparisons a researcher can read to a judgment.




***Build the skeleton first; the Build phase just fills it in.***

ACTIVITY

30 min · groups

# Mock the Deliverable

*Make the spec visible: build the walking skeleton before the build commits you.*

- Ask Claude for three or four distinct shapes of the deliverable in a grid; compare them, then converge on one.
- Build the walking skeleton in HTML: the real shape, with the gold example as the centerpiece and placeholder content elsewhere.
- Check it against the spec (a panel you cannot fill is one you have not specified) and fold the gaps back into the CLAUDE.md.

**OUTPUT** *An HTML walking-skeleton prototype and a copy-as-prompt export: the visible target the Build will fill in.*

GROUP PRESENTATION · ROUND 3

~2 MIN · EACH GROUP

# Show us your HTML mockup

*Open the mockup in a browser. Let us see the shape, not a slide about it.*

- **The walking skeleton:** the real shape, with your gold example as the centerpiece.
- **What leads, what you cut:** what a reader's eye lands on first, and what you left off the screen.
- **One gap it exposed:** something you folded back into the CLAUDE.md after mocking it.

> ***We are listening for a deliverable a stranger could recognize, not a pile of features.***

# Tomorrow you build against the spec

- Day 2 replaces the mocked output with the real thing.
- until the faked gold example becomes a true one.
- because the three artifacts already settled it.
- when the Use week surfaces a surprise, the fix is usually a line, not a refactor.

***Stock the Knowledge · write the living spec · make it visible · then build against it.***